```
# 1
import pandas as pd   # for handling CSVs
import os             # for file paths
import numpy as np    # for handling arrays
import matplotlib.pyplot as plt
import seaborn as sns
folder_path = "/content/"
customers = pd.read_csv(os.path.join(folder_path, "customers.csv"))
marketing = pd.read_csv(os.path.join(folder_path, "marketing.csv"))
orders    = pd.read_csv(os.path.join(folder_path, "orders.csv"))
products  = pd.read_csv(os.path.join(folder_path, "products.csv"))
returns   = pd.read_csv(os.path.join(folder_path, "returns.csv"))
print("Dataframes 'startups', 'services', 'payments', and 'access_logs' loaded successfully!")
```

```
Dataframes 'startups', 'services', 'payments', and 'access_logs' loaded successfully!
```

```
# 3
orders_per_year = orders.groupby("order_year")["order_id"].count().reset_index()
orders_per_month = orders.groupby("order_month")["order_id"].count().reset_index()
orders_per_day = orders.groupby("order_day_of_week")["order_id"].count().reset_index()
print("Orders per Year")
print(orders_per_year)
print("\nOrders per Month")
print(orders_per_month)
print("\nOrders per Day of Week")
print(orders_per_day)
```

```
Orders per Year
   order_year  order_id
0        2022      2634
1        2023      2655
2        2024      2711

Orders per Month
    order_month  order_id
0             1       539
1             2       419
2             3       541
3             4       601
4             5       611
5             6       583
6             7       581
7             8       595
8             9       669
9            10      1023
10           11      1063
11           12       775

Orders per Day of Week
  order_day_of_week  order_id
0            Friday      1130
1            Monday      1125
2          Saturday      1132
3            Sunday      1190
4          Thursday      1137
```

```
5          Tuesday      1150
6        Wednesday      1136
```

```
# 4
orders.rename(columns={"discount_pct": "disc_%"}, inplace=True)
order_products = pd.merge(orders, products, on="product_id", how="left")
order_products["net_revenue"] = (
    order_products["total_amount"] - (order_products["cost_price"] * order_products["quantity"])
)
print(order_products.head())
#
```

```
              order_id   customer_id product_id order_date  quantity  \
0  OD-20230620-BTRR81Q4  RXC-25C63EDB   STN-8466 2023-06-20         1
1  OD-20240918-DDF36MMZ  RXC-AD67D038   GRO-5021 2024-09-18         1
2  OD-20230307-KGS0R0AZ  RXC-DBA2D9A5   ELC-5546 2023-03-07         1
3  OD-20221222-HQD5JNX2  RXC-21E5C222   BTY-1066 2022-12-22         6
4  OD-20221103-V7476TZN  RXC-6FA45D92   TOY-7298 2022-11-03         2

   unit_price  disc_%  discount_amount  total_amount    payment_method  ...  \
0         349      10            34.90        314.10  Cash on Delivery  ...
1         699      15           104.85        594.15               UPI  ...
2        4499       0             0.00       4499.00               UPI  ...
3        1999       0             0.00      11994.00  Cash on Delivery  ...
4         999       0             0.00       1998.00               UPI  ...

  order_day_of_week         product_name     category  cost_price  \
0           Tuesday     Premium Notebook   Stationery         150
1         Wednesday          Quinoa 500g      Grocery         280
2           Tuesday  Keyboard Mechanical  Electronics        2000
3          Thursday         Perfume 50ml       Beauty         800
4          Thursday           Board Game         Toys         400

   selling_price margin_pct stock_qty            supplier  rating  \
0            349       57.0       184  QuickMart Wholesale     3.5
1            699       59.9       190      PrimeDist India     3.7
2           4499       55.5       412    EastGlobe Imports     4.3
3           1999       60.0       440      PrimeDist India     4.0
4            999       60.0       415      GlobalTrade Ltd     3.9

   net_revenue
0       164.10
1       314.15
2      2499.00
3      7194.00
4      1198.00

[5 rows x 25 columns]
```

```
# 5
filtered_orders = orders[
    (orders["order_status"] == "Delivered") &
    (orders["region"] == "South") &
    (orders["order_year"] == 2024)
]
sorted_orders = filtered_orders.sort_values(by="total_amount", ascending=False)
top20_orders = sorted_orders.head(20)
print(top20_orders)
```

```
                order_id   customer_id product_id order_date  quantity  \
5428  OD-20240320-JW67SDSR  RXC-69300FE9   ELC-1563 2024-03-20        10
5045  OD-20240318-VSMWB0CH  RXC-1405A59F   FRN-6433 2024-03-18         4
3255  OD-20240916-0HQTDTG0  RXC-A0EA4EB2   FRN-4975 2024-09-16         4
1338  OD-20240819-5W34CD9J  RXC-FA28CFD4   FRN-8122 2024-08-19         5
7273  OD-20240116-K6V2TTXF  RXC-6A1D9226   HOM-8484 2024-01-16         8
3547  OD-20240401-RZVR5YKG  RXC-4AA00811   FRN-8122 2024-04-01         5
1873  OD-20241121-6DRWVYW2  RXC-3C142531   FRN-6433 2024-11-21         2
447   OD-20240101-ZT4A7SDT  RXC-3074AAD4   ELC-1563 2024-01-01         3
7523  OD-20241215-NG69TVHC  RXC-ADF80E1A   FRN-4664 2024-12-15         5
3350  OD-20240901-W3QDN4QN  RXC-A22DA011   FRN-6433 2024-09-01         1
2741  OD-20241218-Q397GMZG  RXC-3C404432   ELC-8026 2024-12-18         4
5105  OD-20241220-PCRTT3JA  RXC-7BF08004   HOM-7252 2024-12-20         5
6484  OD-20241103-4ER51ZC2  RXC-EE1EC4CC   FRN-6433 2024-11-03         1
3299  OD-20241219-J58R3MQY  RXC-CC7A402C   ELC-8026 2024-12-19         4
720   OD-20240109-E2QZ70VY  RXC-BD5A0B76   FRN-6433 2024-01-09         1
5821  OD-20241116-065DSJ8Y  RXC-1A25792C   TOY-8234 2024-11-16        10
3172  OD-20240128-C73YGB6M  RXC-483452B0   FRN-4664 2024-01-28         3
5585  OD-20240205-ZEB2ZHZ2  RXC-219C53B5   FRN-4664 2024-02-05         3
3658  OD-20241025-D0EEC23D  RXC-E8AC913F   FRN-1082 2024-10-25         5
5736  OD-20240625-2JJ2QQPZ  RXC-DE447A7A   HOM-5966 2024-06-25         8

      unit_price  disc_%  discount_amount  total_amount    payment_method  \
5428       16999      10         16999.00    152991.00            Wallet
5045       34999      10         13999.60    125996.40       Net Banking
3255       18999       0             0.00     75996.00              UPI
1338       13999       0             0.00     69995.00        Debit Card
7273        8999       5          3599.60     68392.40       Net Banking
3547       13999      10          6999.50     62995.50  Cash on Delivery
1873       34999      20         13999.60     55998.40              UPI
447        16999       0             0.00     50997.00              UPI
7523        9499       0             0.00     47495.00              UPI
3350       34999      10          3499.90     31499.10              UPI
2741        7999       5          1599.80     30396.20              UPI
5105        5999       0             0.00     29995.00        Debit Card
6484       34999      15          5249.85     29749.15        Debit Card
3299        7999      10          3199.60     28796.40       Net Banking
720        34999      20          6999.80     27999.20              UPI
5821        3499      20          6998.00     27992.00       Net Banking
3172        9499       5          1424.85     27072.15              UPI
5585        9499       5          1424.85     27072.15       Credit Card
3658        5999      10          2999.50     26995.50              UPI
5736        3499       5          1399.60     26592.40              UPI

     order_status  delivery_days region  order_year  order_month  \
5428    Delivered            5.0  South        2024            3
5045    Delivered            5.0  South        2024            3
3255    Delivered            7.0  South        2024            9
1338    Delivered           12.0  South        2024            8
7273    Delivered            4.0  South        2024            1
3547    Delivered           12.0  South        2024            4
1873    Delivered            8.0  South        2024           11
447     Delivered           12.0  South        2024            1
7523    Delivered            1.0  South        2024           12
3350    Delivered            2.0  South        2024            9
2741    Delivered            2.0  South        2024           12
5105    Delivered            8.0  South        2024           12
6484    Delivered            2.0  South        2024           11
```

```
# 6
category_summary = (
    order_products.groupby("category")
    .agg(
        total_revenue=("net_revenue", "sum"),
        order_count=("order_id", "count"),
        avg_order_value=("total_amount", "mean"),
        max_order=("total_amount", "max")
    )
    .reset_index()
)
category_summary = category_summary.sort_values(by="total_revenue", ascending=False)
print(category_summary.head(20))
```

```
         category  total_revenue  order_count  avg_order_value  max_order
4       Furniture    10079821.35          668     28166.648728   314991.0
3     Electronics     3816862.00          804      9456.731343   152991.0
6  Home & Kitchen     3327306.65          825      7556.795939    71992.0
2        Clothing     1594587.75          820      3357.753354    42740.5
9            Toys     1555659.10          872      3204.138876    29741.5
7          Sports     1293295.20          804      2862.307463    25192.8
0          Beauty     1120202.25          845      2350.547041    19990.0
5         Grocery      719706.10          853      1489.889918     9992.0
1           Books      634912.15          850      1329.755471     6990.0
8      Stationery      425771.70          659      1165.556449     9990.0
```

```
# 7
orders_customers = pd.merge(orders, customers, on="customer_id", how="left")
master_df = pd.merge(orders_customers, products, on="product_id", how="left")
master_df = master_df[master_df["order_status"] == "Delivered"]
print("Row count (Delivered orders):", master_df.shape[0])
```

```
Row count (Delivered orders): 5821
```

```
# 8
customers["first_name"] = customers["customer_name"].str.split().str[0]
customers["initials"] = (
    customers["customer_name"].str.split().str[0].str[0] + "." +
    customers["customer_name"].str.split().str[-1].str[0] + "."
)
city_counts = customers["city"].value_counts()
print(city_counts)
```

```
city
Hyderabad       141
Nagpur          131
Chennai         122
Indore          122
Jaipur          120
Bhopal          118
Surat           117
Chandigarh      114
Ahmedabad       110
Mumbai          108
Visakhapatnam   106
Kolkata         105
```

```
Lucknow          102
Coimbatore       101
Pune             100
Kochi             97
Bengaluru         95
Delhi             91
Name: count, dtype: int64
```

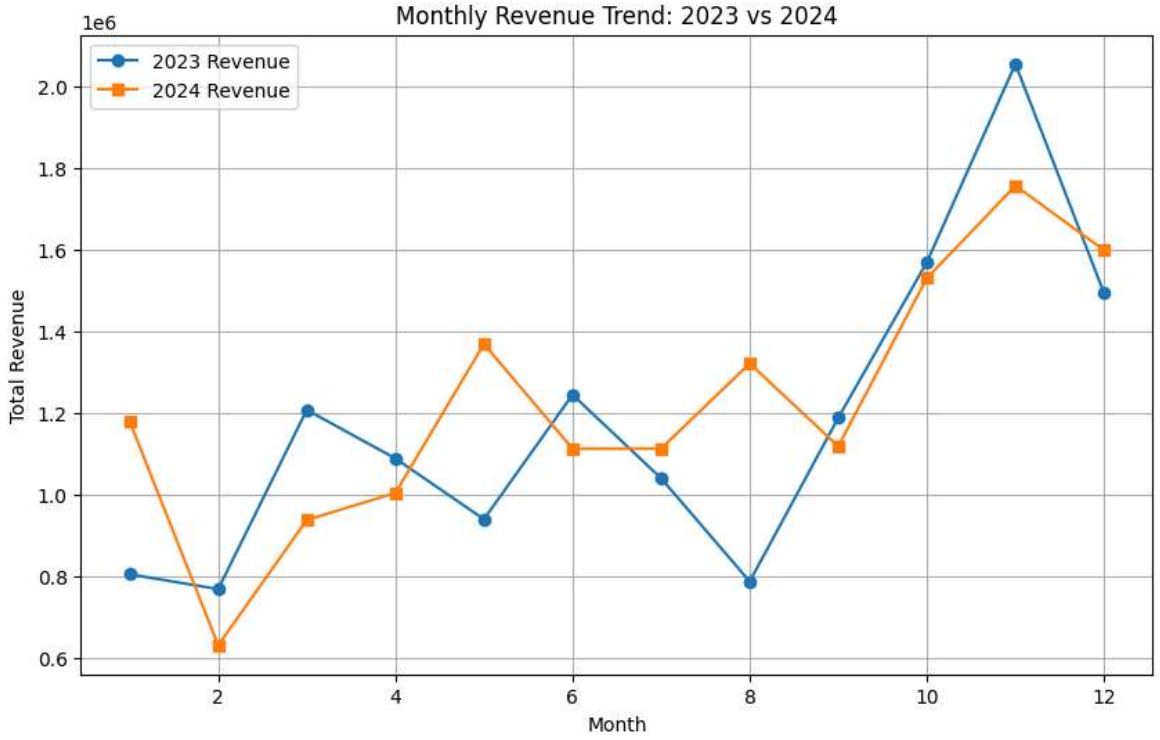
```
# 9
median_delivery = orders.loc[orders["order_status"] == "Delivered", "delivery_days"].median()
orders["delivery_days"] = orders["delivery_days"].fillna(median_delivery)
null_count = orders["delivery_days"].isnull().sum()
print("Remaining NULLs in delivery_days:", null_count)
```

```
Remaining NULLs in delivery_days: 0
```

```
# 10
mean_val = orders["total_amount"].mean()
std_val  = orders["total_amount"].std()
print("Mean:", mean_val)
print("Std Dev:", std_val)
orders["z_score"] = (orders["total_amount"] - mean_val) / std_val
orders["is_outlier"] = orders["z_score"].abs() > 3
outlier_count = orders["is_outlier"].sum()
print("Number of outliers:", outlier_count)
```

```
Mean: 5707.12928125
Std Dev: 13587.968113170553
Number of outliers: 154
```

```
# 11
import pandas as pd
import matplotlib.pyplot as plt
orders["order_date"] = pd.to_datetime(orders["order_date"])
orders["year"] = orders["order_date"].dt.year
orders["month"] = orders["order_date"].dt.month
orders_filtered = orders[orders["year"].isin([2023, 2024])]
monthly_revenue = (
    orders_filtered.groupby(["year", "month"])
    ["total_amount"]
    .sum()
    .reset_index()
)
pivot_rev = monthly_revenue.pivot(index="month", columns="year", values="total_amount")
plt.figure(figsize=(10,6))
plt.plot(pivot_rev.index, pivot_rev[2023], marker="o", label="2023 Revenue")
plt.plot(pivot_rev.index, pivot_rev[2024], marker="s", label="2024 Revenue")
plt.title("Monthly Revenue Trend: 2023 vs 2024")
plt.xlabel("Month")
plt.ylabel("Total Revenue")
plt.legend()
plt.grid(True)
plt.savefig("monthly_revenue_trend.png", dpi=300)
plt.show()
```

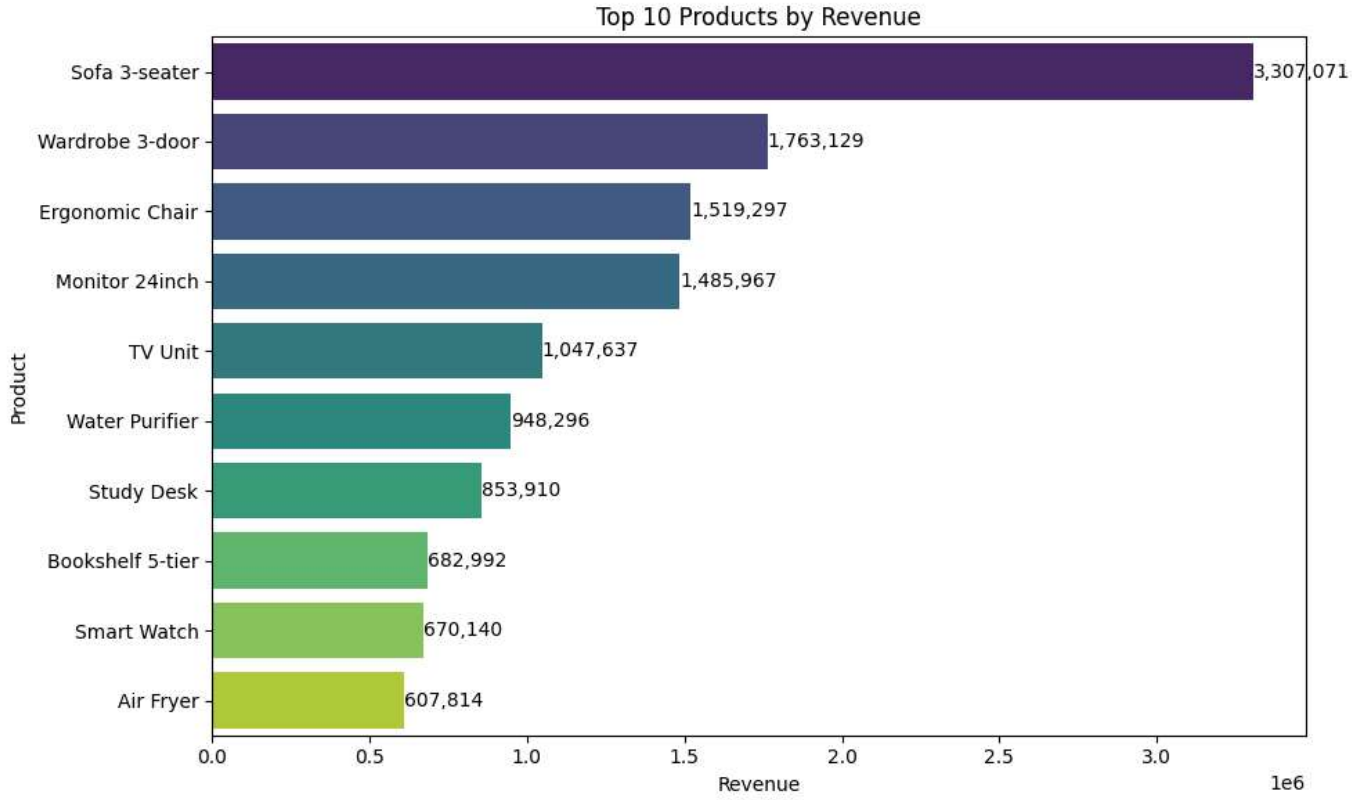



```
# 12
import seaborn as sns
import matplotlib.pyplot as plt
top_products = (
    order_products.groupby("product_name")["net_revenue"]
    .sum()
    .reset_index()
    .sort_values(by="net_revenue", ascending=False)
    .head(10)
)
plt.figure(figsize=(10,6))
barplot = sns.barplot(
    x="net_revenue",
    y="product_name",
    data=top_products,
    palette="viridis"
)
for index, value in enumerate(top_products["net_revenue"]):
    plt.text(value + 50, index, f"{value:,.0f}", va="center")
plt.title("Top 10 Products by Revenue")
plt.xlabel("Revenue")
plt.ylabel("Product")
plt.tight_layout()
plt.show()
```

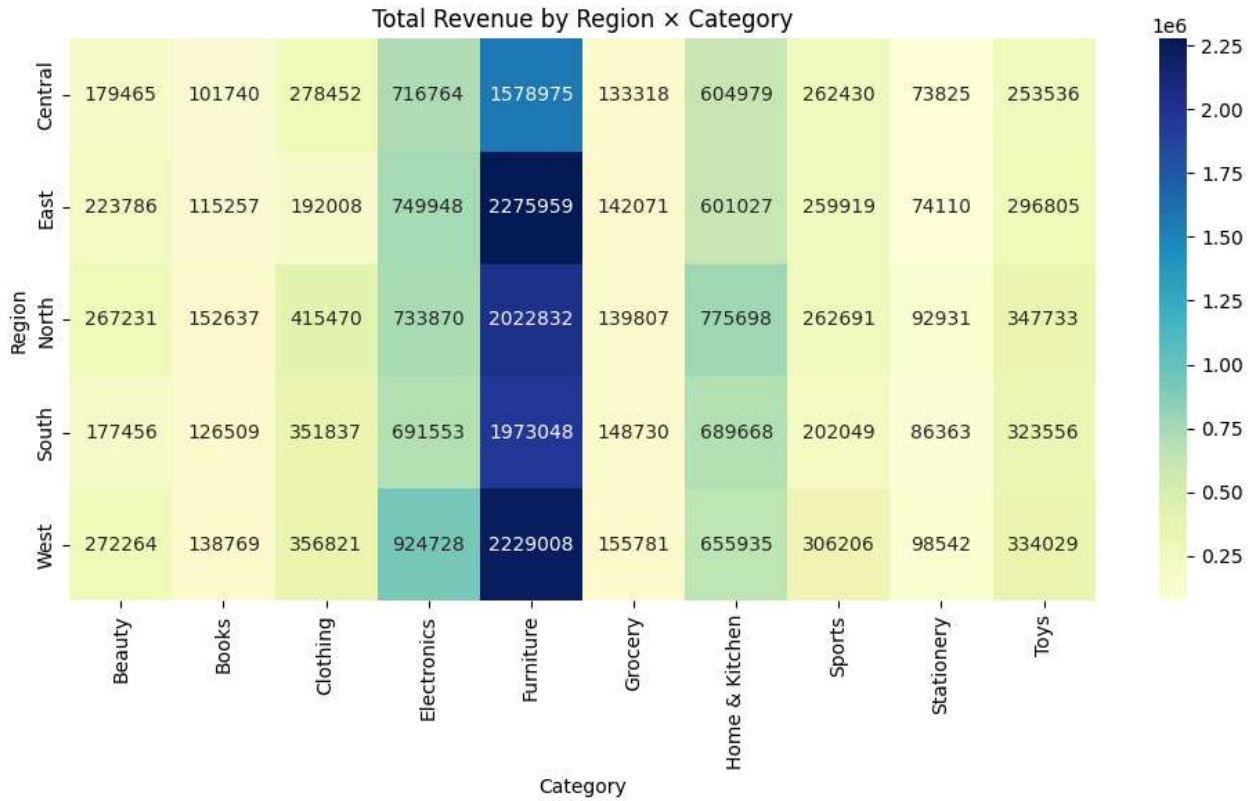
```
/tmp/ipykernel_2902/4029587292.py:15: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

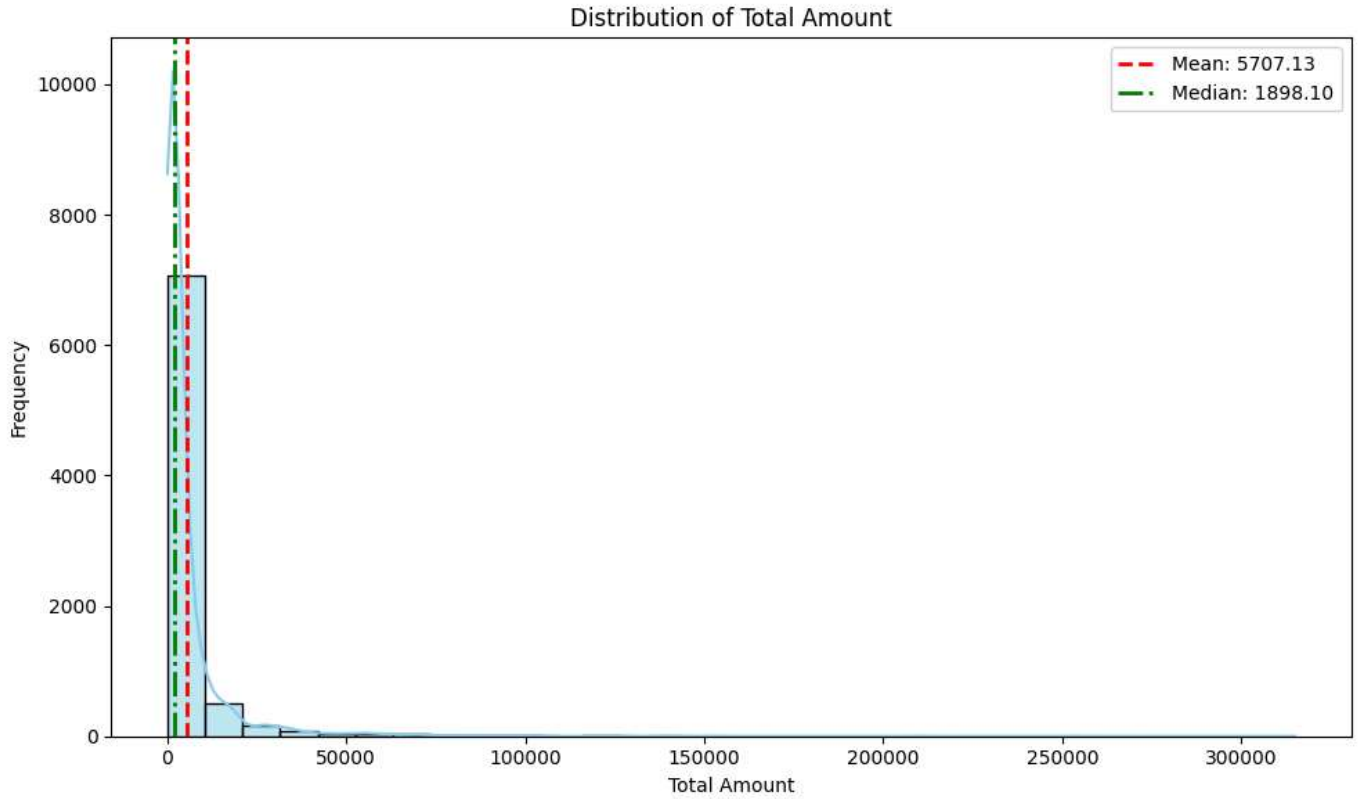
  barplot = sns.barplot(
```




```
# 13
import seaborn as sns
import matplotlib.pyplot as plt
pivot_rev = order_products.pivot_table(
    index="region",
    columns="category",
    values="net_revenue",
    aggfunc="sum",
    fill_value=0
)
plt.figure(figsize=(10,6))
sns.heatmap(pivot_rev, annot=True, fmt=".0f", cmap="YlGnBu")
plt.title("Total Revenue by Region × Category")
plt.xlabel("Category")
plt.ylabel("Region")
plt.tight_layout()
plt.show()
```




```
# 14
import matplotlib.pyplot as plt
import seaborn as sns
mean_val = orders["total_amount"].mean()
median_val = orders["total_amount"].median()
plt.figure(figsize=(10,6))
sns.histplot(orders["total_amount"], bins=30, kde=True, color="skyblue")
plt.axvline(mean_val, color="red", linestyle="--", linewidth=2, label=f"Mean: {mean_val:.2f}")
plt.axvline(median_val, color="green", linestyle="-.", linewidth=2, label=f"Median: {median_val:.2f}")
plt.title("Distribution of Total Amount")
plt.xlabel("Total Amount")
plt.ylabel("Frequency")
plt.legend()
plt.tight_layout()
plt.show()
```

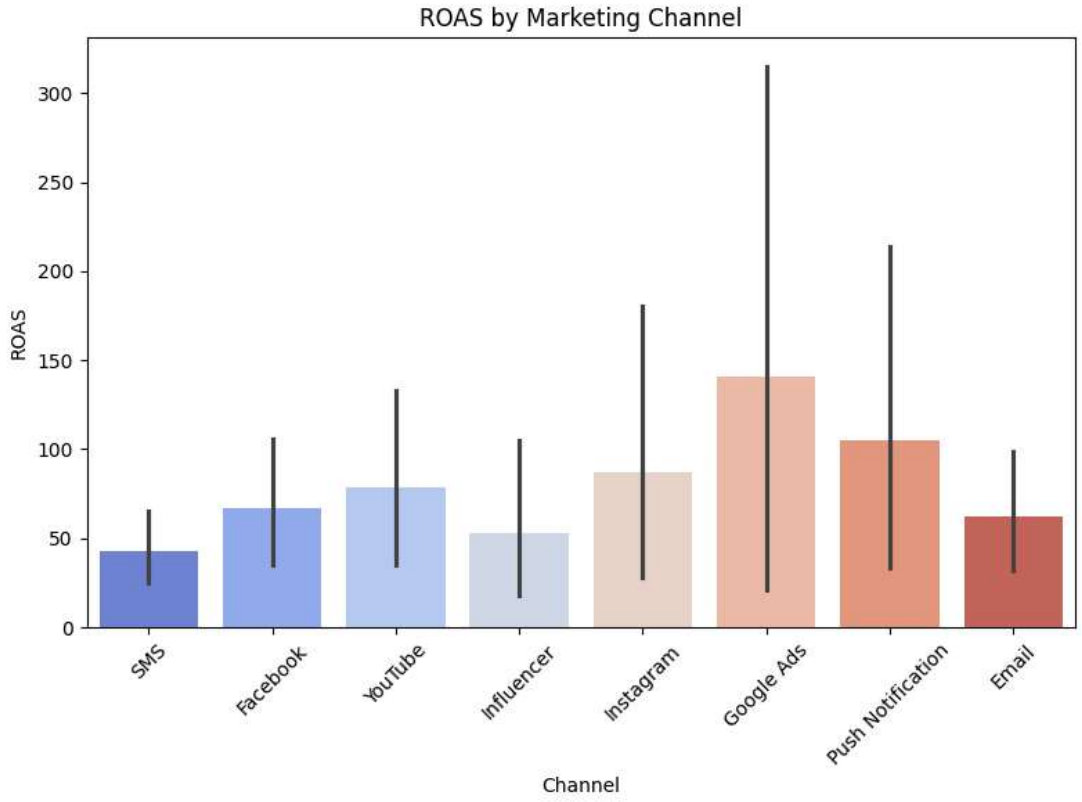



```
# 15
marketing["CTR"] = marketing["clicks"] / marketing["impressions"]
marketing["CVR"] = marketing["conversions"] / marketing["clicks"]
marketing["ROAS"] = marketing["revenue_generated"] / marketing["spend_inr"]
top_channels = marketing.sort_values(by="ROAS", ascending=False).head(3)
print("Top 3 Channels by ROAS:")
print(top_channels[["channel", "ROAS"]])
import seaborn as sns
import matplotlib.pyplot as plt
plt.figure(figsize=(8,6))
sns.barplot(x="channel", y="ROAS", data=marketing, palette="coolwarm")
plt.title("ROAS by Marketing Channel")
plt.xlabel("Channel")
plt.ylabel("ROAS")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
Top 3 Channels by ROAS:
        channel         ROAS
42   Google Ads  1543.033681
107  Google Ads  1494.512103
116   Instagram  1396.192723
/tmp/ipykernel_2902/3093187603.py:13: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(x="channel", y="ROAS", data=marketing, palette="coolwarm")
```




```
# 16
import pandas as pd
orders_customers = pd.merge(orders, customers, on="customer_id", how="left")
orders_customers["registration_date"] = pd.to_datetime(orders_customers["registration_date"])
orders_customers["order_date"] = pd.to_datetime(orders_customers["order_date"])
orders_customers["registration_month"] = orders_customers["registration_date"].dt.to_period("M")
orders_customers["order_month"] = orders_customers["order_date"].dt.to_period("M")
cohort_matrix = pd.pivot_table(
    orders_customers,
    index="registration_month",
    columns="order_month",
    values="customer_id",
    aggfunc="nunique",
    fill_value=0
```

```
)
print(cohort_matrix.head(5))
```

```
order_month         2022-01  2022-02  2022-03  2022-04  2022-05  2022-06  \
registration_month
2021-01                   4        8        3        6        9        1
2021-02                   6        7        4        2        3        0
2021-03                   4        3        4        5        5        2
2021-04                   7        3        6        5        7        3
2021-05                   8        5        9        5        7        8

order_month         2022-07  2022-08  2022-09  2022-10  ...  2024-03  2024-04  \
registration_month                                      ...
2021-01                   2        7        4        8  ...        3        4
2021-02                   6        4        5       11  ...        2        5
2021-03                   4        7        5       12  ...        4        8
2021-04                   4        4        1        8  ...        1        7
2021-05                   6        4        4       14  ...        4        5

order_month         2024-05  2024-06  2024-07  2024-08  2024-09  2024-10  \
registration_month
2021-01                   6       10        5        7        4        5
2021-02                   4        3       10        5        4        8
2021-03                   9        5        6        5        2        4
2021-04                   4        6        2       10        5       11
2021-05                  10        4        8       13        6       12

order_month         2024-11  2024-12
registration_month
2021-01                  15        7
2021-02                   6        1
2021-03                  11       10
2021-04                   6        9
2021-05                  11       13

[5 rows x 36 columns]
```

```
# 17
control_mean = marketing.loc[marketing["ab_group"] == "Control", "ROAS"].mean()
treatment_mean = marketing.loc[marketing["ab_group"] == "Treatment", "ROAS"].mean()
diff = treatment_mean - control_mean
print("Control Avg ROAS:", control_mean)
print("Treatment Avg ROAS:", treatment_mean)
print("Difference (Treatment - Control):", diff)
from scipy import stats
control_roas = marketing.loc[marketing["ab_group"] == "Control", "ROAS"]
treatment_roas = marketing.loc[marketing["ab_group"] == "Treatment", "ROAS"]
t_stat, p_val = stats.ttest_ind(treatment_roas, control_roas, equal_var=False)
print("t-statistic:", t_stat)
print("p-value:", p_val)
```

```
Control Avg ROAS: 60.36948627536181
Treatment Avg ROAS: 96.12354780740155
Difference (Treatment - Control): 35.75406153203974
t-statistic: 1.2641668877880063
p-value: 0.20768539825657203
```